

GitDash Project
Project Proposal
Version <2.0>

By OSP_Team_3

Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

Revision History

Date	Version	Description	Author
<4/29/25>	<1.0>	First writing	김민규
<5/1/25>	<2.0>	Write Details	김민규

Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

Table of Contents

1.	Introduction.....	4
1.1	Purpose.....	4
1.2	Definitions, Acronyms, and Abbreviations.....	4
1.3	References.....	4
2.	Specific Requirements.....	5
2.1	Functionally.....	5
2.1.1	Dashboard Overlay Rendering.....	5
2.1.2	Key Metrics Summary.....	5
2.1.3	Data Visualizataion.....	5
2.1.4	Recent Activities List.....	5
2.2	Non-Functionally.....	5
2.2.1	Clean Code & Architecture.....	5
2.2.2	Backend Data Processing.....	6
2.2.3	Containerization.....	6
3.	Supporting Information.....	7
3.1	System Architecture.....	7
3.2	Technology Stack.....	7
3.3	Project Plan & Task Assignments.....	8
3.4	Role Assignments.....	8

Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

Project Proposal

1. Introduction

1.1 Purpose

The primary purpose of this project is to develop a lightweight Chrome browser extension that provides an instant, highly visual analytics dashboard for GitHub repositories. Currently, developers and project managers must navigate through multiple fragmented pages to assess a repository's overall health, contributor activity, and the status of recent issues and pull requests.

This project aims to alleviate this friction by injecting a seamless dashboard overlay directly into the native GitHub interface. By consolidating and visualizing key project metrics—such as commit trends, top contributors, and activity heatmaps—the extension will significantly enhance data accessibility and the overall user experience. Furthermore, from a technical perspective, the project aims to design a highly optimized backend architecture utilizing NestJS and Redis caching to effectively mitigate GitHub API rate limits and ensure fast, reliable data delivery.

1.2 Definitions, Acronyms, and Abbreviations

PR	Pull Request
DOM	Document Object Model
BFF	Backend For Frontend
Rate Limit	Restriction on the number of API calls

1.3 References

[GitHub REST/GraphQL API](#) Documentation

[Chart.js Official Documentation](#)

[NestJS Official Documentation](#)

Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

2. Specific Requirements

2.1 Functionally

2.1.1 Dashboard Overlay Rendering

- R1.1 The System shall provide a 'Dashboard' button in the top right corner upon entering
- R1.2 The system shall provide an overlay UI feature that allows users to open and close the dashboard panel by the button

2.1.2 Key Metrics Summary

- R2.1 The system shall summarize and provide the total number of commits based on the last 30 days (or a specific period).
- R2.2 The system shall calculate and provide the number of project contributors within the specific period.
- R2.3 The system shall provide the number of closed issues and merged PRs in the form of top widgets

2.1.3 Data Visualizataion

- R3.1 The System shall visualize the commit trend over time as a Line Chart.
- R3.2 The system shall visualize the ratio of top users with high contributions as a Doughnut Chart.
- R3.3 The system shall render the development activity by day of the week and week as a Heatmap.
- R3.4 The system shall provide issue status (Open/Closed) and language distribution in the form of progress bars and charts.

2.1.4 Recent Activities List

- R4.1 The system shall display the list of recently merged PRs in a text list format.
- R4.2 The system shall provide a list of recently updated major issues.

2.2 Non-Functionally

2.2.1 Clean Code & Architecture

- R5.1 The frontend system shall be written in JS.
- R5.2 The system shall strictly separate modules by function, such as communication (api.js), DOM manipulation (content.js), and visualization (chart.js), to reduce code complexity

Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

2.2.2 Backend Data Processing

- R6.1 The backend system shall apply the BFF (Backend For Frontend) pattern to refine data in order to reduce the computational burden on the frontend.
- R6.2 The backend system shall implement a caching layer utilizing Redis to defend against the GitHub API's Rate Limit.

2.2.3 Containerization

- R6.1 The backend application and database (PostgreSQL, Redis) shall be containerized and deployed using Docker and docker-compose.

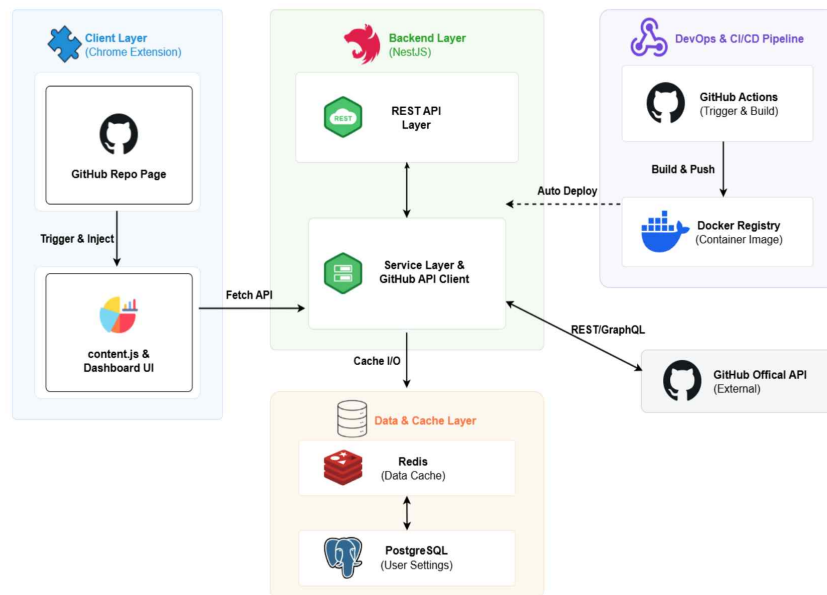
Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

3. Supporting Information

3.1 System Architecture

- Client Layer : Consists of a Chrome Extension centered on Vanilla JS, analyzing the GitHub webpage DOM to inject the UI.
- Backend Layer : Built as a single REST API server based on NestJS to call the GitHub API and process data.
- Data & Cache Layer : Consists of PostgreSQL (saving user settings) and Redis (caching responses and defending against rate limits).
- DevOps Layer : Applies an automated pipeline for CI/CD setup and Docker container deployment based on GitHub Actions.

System Architecture Diagram



3.2 Technology Stack

Category	Framework
Frontend	HTML, Css, JS
Data Visualization	Chart.js
Backend	NestJS
Database	PostgreSQL
Cache	Redis
DevOps	Docker, GitHub Actions
Collaboration	GitHub (Issues, PRs, Projects)

Gitdash Project	Version: <2.0>
Project Proposal	Date: <5/1/25>

3.3 Project Plan & Task Assignments

- Week 1 (5/1 - 5/8) : Requirement setup, Docker local environment configuration, plugin manifest setup, and CI/CD connection.
- Week 2 (5/9 - 5/16) : Backend API design, DB schema construction, frontend module design, and DOM analysis.
- Week 3 (5/17 - 5/24) : Frontend/Backend communication integration, UI injection script writing, and Chart.js data visualization application.
- Week 4 (5/25 - 5/31) : Redis caching application, UI enhancements such as dark mode, integrated QA, and bug fixes.
- Week 5 (6/1 - 6/5): Final testing, refactoring, and preparation of presentation materials for the final presentation on Tuesday, June 9th.

3.4 Role Assignments

- PM 김민규: Project overview, GitHub Projects Kanban management, schedule adjustment, peer review management, and presentation material creation.
- Backend Dev 하강민 : NestJS API development, DB/Redis integration, Docker image creation, and CI/CD deployment.
- Frontend Dev 장팅웨이 : Chrome Extension implementation, DOM control and UI injection, and visualization component development utilizing Chart.js.